

Java I/O (NIO)

doc. dr Vladimir Jocović

15.06.2026.

Fajl sistem i putanja

- Fajl sistem organizuje datoteke na disku u hijerarhijsku strukturu (stablo).
- Na vrhu te strukture nalazi se koreni direktorijum (root).
- Direktorijumi mogu da sadrže datoteke i druge direktorijume (rekurzija).
- Svaka datoteka se identifikuje preko putanje (path), koja opisuje njenu lokaciju u hijerarhiji počevši od korenog direktorijuma.
- Putanje mogu da budu:
 - Apsolutne – počinju od root direktorijuma i opisuju punu lokaciju (npr. C:\Users\John\file.txt ili /home/john/file.txt).
 - Relativne – zavise od trenutnog konteksta i same po sebi nisu potpune (neophodno je da se kombinuju sa drugom putanjom u cilju pristupa datoteci).
- U Java stringovima znak \ ima posebno značenje (escape karakter), pa za Windows putanje moraju da se pišu dva znaka \, tj. \\ da bi se dobio jedan stvarni *backslash* u putanji.

Putanja

- Klasa Path predstavlja putanju do datoteke ili direktorijuma.
- Klasa Path je deo novijeg **java.nio.file** paketa (Java 7+) za rad sa datotekama.
- Oblik putanje zavisi od operativnog sistema.
 - Različiti operativni sistemi koriste različite separatore (Windows -> \, Linux i macOS -> /).
 - Java mora da podrži sve slučajeve pa zato koristi Path apstrakciju.
- Putanja ne mora da pokazuje na postojeću datoteku.
- Putanja se koristi isključivo za predstavljanje lokacije datoteke, a ne za rad sa njenim sadržajem.
- Klasa Files sadrži operacije koje omogućavaju rad nad datotekama.
 - Takođe je deo **java.nio.file** paketa.

Putanja

- Path se kreira preko Paths.get() poziva.
- Putanja koja se zadaje može da bude apsolutna ili relativna.

```
Path p = Paths.get("C:\\Users\\John\\file.txt");
```

```
System.out.println(p); // prints: C:\Users\John\file.txt
```

- Mogu da se dohvate nazivi: datoteke, roditeljskog direktorijuma i korenog direktorijuma.

```
Path p = Paths.get("C:\\Users\\John\\file.txt");
```

```
System.out.println(p.getFileName()); // prints: file.txt
```

```
System.out.println(p.getParent()); // prints: C:\Users\John
```

```
System.out.println(p.getRoot()); // prints: C:\
```

Putanja

- Može da se izvrši normalizacija putanje.

```
Path p = Paths.get("C:\\Users\\.\\.\\John\\.\\.\\file.txt");  
System.out.println(p.normalize()); // prints: C:\Users\file.txt
```

- Putanja može da se pretvori u apsolutnu putanju i URI (Uniform Resource Identifier – jedinstvena identifikacija imena ili lokacije resursa na internetu).

```
Path p = Paths.get("file.txt");  
// below code prints: C:\Users\John\project\file.txt  
System.out.println(p.toAbsolutePath());  
// below code prints: file:///C:/Users/John/project/file.txt  
System.out.println(p.toUri());
```

Putanja

- Može da se doda podputanja na postojeću putanju.

```
Path base = Paths.get("C:\\Users\\John");
```

```
Path full = base.resolve("file.txt");
```

```
System.out.println(full); // prints: C:\Users\John\file.txt
```

- Moguće je da se napravi put od jedne lokacije do druge.

```
Path p1 = Paths.get("C:\\Users\\John");
```

```
Path p2 = Paths.get("C:\\Users\\John\\Documents\\file.txt");
```

```
System.out.println(p1.relativeTo(p2)); // prints: Documents\file.txt
```

Putanja

- Može da se proveriti početak, kraj i jednakost putanje.

```
Path p = Paths.get("C:\\Users\\John\\file.txt");  
System.out.println(p.startsWith("C:\\Users")); // prints: true  
System.out.println(p.endsWith("file.txt")); // prints: true  
System.out.println(p.equals(Paths.get("C:\\Users\\z\\a.txt"))); // prints: false
```

- Putanja može da se posmatra kao niz delova.

```
Path p = Paths.get("C:\\Users\\John\\file.txt");  
// below code prints: Users_John_file.txt  
for (Path part : p) {  
    System.out.print(part + "_");  
}
```

Operacije nad datotekama

- Files je glavna klasa za rad sa datotekama (čitanje, pisanje, itd.).
- Metode klase Files su statičke i rade nad Path objektima.

```
Path path = Paths.get("test.txt");
```

```
// assuming file test.txt exists
```

```
System.out.println(Files.exists(path)); // prints: true
```

- Resursi poput tokova treba da budu zatvoreni nakon korišćenja.
- Try-with-resources automatski zatvara resurse i preporučeni je način rada od Jave 7.

```
try (BufferedReader br = Files.newBufferedReader(Paths.get("test.txt"))) {  
    System.out.println(br.readLine()); // reads one line  
}
```


Datoteke i izuzeci

- Operacije nad datotekama mogu da izazovu IOException.
- Do greške može da dođe usled različitih nevalidnih situacija u toku rada sa datotekom.
 - Npr. otvaranje nepostojeće datoteke ili nedozvoljen pristup.
- Izuzeci se obrađuju pomoću try-catch bloka.

```
try {  
    Files.delete(Paths.get("abc.txt"));  
} catch (IOException e) {  
    System.out.println(e.getMessage());  
}
```

Postojanje datoteka

- `Files.exists()` proverava da li datoteka ili direktorijum postoje.
- `Files.notExists()` proverava da li datoteka ili direktorijum ne postoje.
- Pozivi `!Files.exists(path)` i `Files.notExists(path)` nisu ekvivalentni.
 - Ako nema dozvole pristupa datoteci, oba poziva `Files.exists(path)` i `Files.notExists(path)` mogu da vrate `false`. Ako oba poziva vrate `false`, postojanje datoteke ne može da se potvrdi.

```
Path path = Paths.get("test.txt");
```

```
System.out.println(Files.exists(path));
```

```
System.out.println(Files.notExists(path));
```

Ispitivanje svojstva datoteka

- `Files.isReadable()` proverava da li datoteka može da se čita.
- `Files.isWritable()` proverava da li datoteka može da se piše.
- `Files.isExecutable()` proverava da li datoteka može da se izvršava.

```
Path file = Paths.get("test.txt");
```

```
System.out.println(Files.isReadable(file));
```

```
System.out.println(Files.isWritable(file));
```

```
System.out.println(Files.isExecutable(file));
```

- `Files.isSameFile()` proverava da li dve putanje (Path) vode do iste datoteke.

```
Path p1 = Paths.get("C:\\Users\\John\\file.txt");
```

```
Path p2 = Paths.get("C:\\Users\\John\\Docs\\..\\file.txt");
```

```
System.out.println(Files.isSameFile(p1, p2)); // prints: true
```

- Ove metode rade i za datoteke i za direktorijume, ali značenje zavisi od operativnog sistema.

Brisanje datoteka

- Files klasa omogućava brisanje datoteka i direktorijuma.
 - Direktorijum mora da bude prazan da bi se obrisao.
- Files.delete() briše datoteku ili direktorijum.
 - Emituje se izuzetak u slučaju nevalidne situacije.

```
Path path = Paths.get("test.txt");
try {
    Files.delete(path);
} catch (NoSuchFileException e) {
    System.out.println("Datoteka ne postoji!");
} catch (DirectoryNotEmptyException e) {
    System.out.println("Direktorijum nije prazan!");
} catch (IOException e) {
    System.out.println("Greška: " + e.getMessage());
}
```

Brisanje datoteka

- `Files.deleteIfExists()` briše datoteku, ako ona postoji.
 - Ako datoteka ne postoji, ne radi ništa i ne emituje izuzetak `NoSuchFileException`, za razliku od `delete()` metoda.

```
Path path = Paths.get("test.txt");
try {
    Files.deleteIfExists(path);
} catch (IOException e) {
    System.out.println("Greška: " + e.getMessage());
}
```

Kopiranje datoteka

- `Files.copy()` kopira datoteku ili direktorijum.
- Prilikom kopiranja direktorijuma ne kopira se njegov sadržaj (rezultat je prazan direktorijum).
- U slučaju datoteke, kopira se njen sadržaj na novu lokaciju.
 - Greška je, ako cilj već postoji, osim ako se ne kaže drugačije.

```
Path source = Paths.get("a.txt");
```

```
Path target = Paths.get("b.txt");
```

```
Files.copy(source, target);
```

- Moguće je i da se dozvoli prepisivanje postojeće datoteke.

```
Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);
```

Premeštanje datoteka

- `Files.move()` premešta datoteku ili direktorijum (original nestaje).
- Prilikom premeštanja direktorijuma premešta se ceo direktorijum zajedno sa njegovim sadržajem.
- Premeštanje može da znači i preimenovanje, pored relokacije.

```
Path source = Paths.get("a.txt");
```

```
Path target = Paths.get("folder/a.txt");
```

```
Files.move(source, target);
```

- Moguće je i da se dozvoli prepisivanje postojeće datoteke.

```
Files.move(source, target, StandardCopyOption.REPLACE_EXISTING);
```

Čitanje i pisanje tekstualnih datoteka

- Male tekstualne datoteke mogu da se čitaju korišćenjem `Files.readAllLines()`, čime se učitava cela datoteka u memoriju.

```
Path path = Paths.get("test.txt"); // test.txt contains lines: Hello\nWorld
List<String> lines = Files.readAllLines(path);
System.out.println(lines); // prints [Hello, World]
```

- Male tekstualne datoteke mogu da se pišu korišćenjem `Files.write()`.

```
Path path = Paths.get("output.txt");
List<String> lines = Arrays.asList("Hello", "World");
Files.write(path, lines); // test.txt contains lines: Hello\nWorld
```


Čitanje i pisanje tekstualnih datoteka

- Velike tekstualne datoteke mogu da se čitaju (red po red teksta) korišćenjem klase `BufferedReader`.

```
Path path = Paths.get("test.txt");
try (BufferedReader br = Files.newBufferedReader(path)) {
    String line;
    while ((line = br.readLine()) != null) { System.out.println(line); }
}
```

- Velike tekstualne datoteke mogu da se pišu korišćenjem klase `BufferedWriter`.

```
Path path = Paths.get("log.txt");
try (BufferedWriter bw = Files.newBufferedWriter(path)) {
    bw.write("Hello"); bw.newLine();
    bw.write("World");
}
```

Čitanje i pisanje binarnih datoteka

- Male binarne datoteke mogu da se čitaju korišćenjem `Files.readAllBytes()` za učitavanje svih bajtova datoteke u memoriju.

```
Path path = Paths.get("image.png");
```

```
byte[] data = Files.readAllBytes(path);
```

```
System.out.println(data.length);
```

- Male binarne datoteke mogu da se pišu korišćenjem `Files.write()`.

```
byte[] data = new byte [100];
```

```
Path path = Paths.get("file.abc");
```

```
Files.write(path, data);
```

Čitanje i pisanje binarnih datoteka

- Velike binarne datoteke mogu da se čitaju korišćenjem implementacije klase `InputStream`.
 - Jedna od implementacija klase `InputStream` jeste klasa `java.io.FileInputStream`.
 - Ne učitava se cela datoteka u memoriju, već se čita deo po deo.

```
Path path = Paths.get("video.mp4");
try (InputStream in = Files.newInputStream(path)) {
    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = in.read(buffer)) != -1) { System.out.println("Read: " + bytesRead); }
}
```

- Za pisanje velikih binarnih datoteka koristi se klasa `OutputStream`.
 - Jedna od implementacija klase `OutputStream` jeste klasa `java.io.FileOutputStream`.

```
Path path = Paths.get("copy.bin");
byte[] data = new byte[1024];
try (OutputStream out = Files.newOutputStream(path)) {
    out.write(data);
}
```

Čitanje i pisanje datoteka

- Prilikom pokušaja čitanja datoteka, ona mora da postoji.
 - U suprotnom, emituje se izuzetak `NoSuchFileException`.

```
BufferedReader br = Files.newBufferedReader(path);
```

```
InputStream in = Files.newInputStream(path);
```

- Prilikom pisanja datoteke može da se definiše način otvaranja datoteke korišćenjem `OpenOption` modova.

```
// If file does not exist, it will be created, else new data will be appended (file not erased).
```

```
BufferedWriter bw = Files.newBufferedWriter(path, CREATE, APPEND);
```

```
// If file does not exist, it will be created, else new data will be truncated (file is erased).
```

```
OutputStream out = Files.newOutputStream(path, CREATE, TRUNCATE_EXISTING);
```

- Oba prethodna poziva kreiraju novu datoteku, ako ona ne postoji.
Kod `APPEND` novi podaci se dodaju na kraj postojeće datoteke.
Kod `TRUNCATE_EXISTING` postojeći sadržaj se briše pre upisa novih podataka.

Kreiranje direktorijuma

- Direktorijum se kreira pomoću `Files.createDirectory()`.
 - Roditeljski direktorijum mora već da postoji.
 - Ukoliko roditeljski direktorijum ne postoji, emituje se izuzetak `NoSuchFileException`.

// If C:\temp exists, creates C:\temp\test, else `NoSuchFileException`.

```
Path dir = Paths.get("C:\\temp\\test");
```

```
Files.createDirectory(dir);
```

- Kreiranje direktorijuma, ukoliko roditeljski direktorijum ne postoji, obavlja se pomoću `Files.createDirectories()`.

// Directories C:\temp, C:\temp\java, C:\temp\java\test will be created
// if they do not exist.

```
Path dir = Paths.get("C:\\temp\\java\\test");
```

```
Files.createDirectories(dir);
```

Listanje sadržaja direktorijuma

- Prolazak kroz sadržaj direktorijuma moguć je pomoću `Files.newDirectoryStream()`.
 - Poziv vraća i poddirektorijume i datoteke.
 - Ne učitava se ceo sadržaj odjednom u memoriju.
 - Koristi se uz `try-with-resources` konstrukt.

// C:\temp directory contains: test.txt, image.png, docs; the following code prints:

// test.txt

// image.png

// docs

```
Path dir = Paths.get("C:\\temp");
```

```
try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir)) {  
    for (Path p : stream) { System.out.println(p.getFileName()); }  
}
```

Filtriranje sadržaja direktorijuma

- Moguće je filtrirati sadržaj direktorijuma.
- Često se koristi za pronalaženje datoteka određenog tipa.
- Glob obrasci predstavljaju jednostavan način za pretragu datoteka prema šablonu naziva.
 - Znak * predstavlja proizvoljan broj karaktera (npr. *.txt odgovara: a.txt, test.txt, document.txt).
 - Znak ? predstavlja tačno jedan karakter (npr. ?.txt odgovara: a.txt, b.txt, ali ne i document.txt).

// If directory C:\temp contains: a.txt, b.txt, c.png, d.pdf; the following code prints:

// a.txt

// b.txt

```
Path dir = Paths.get("C:\\temp");
```

```
try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir, "*.txt")) {
```

```
    for (Path p : stream) { System.out.println(p.getFileName()); }
```

```
}
```

Rekurzivni obilazak direktorijuma

- Rekurzivni obilazak direktorijuma moguć je pomoću `Files.walkFileTree()`.
- Koristi se za rad nad celim stablom datoteka.
- Obilazak stabla se vrši po dubini (depth-first).
- Implementira se interfejs `FileVisitor` ili se proširuje `SimpleFileVisitor` klasa.
 - Oba tipa su parametrizovani tipovi (generici).
- Interfejs `FileVisitor` ima 4 metoda:
 - `preVisitDirectory()` – poziva se pre ulaska u direktorijum pri obilasku stabla.
 - `visitFile()` – poziva se prilikom posete datoteke pri obilasku stabla.
 - `postVisitDirectory()` – poziva se nakon izlaska iz direktorijuma pri obilasku stabla.
 - `visitFileFailed()` – poziva se, ukoliko postoji greška u pristupu (npr. pristup datoteci).
- Najčešće se koristi klasa `SimpleFileVisitor` jer onda ne moraju da se implementiraju sve metode interfejsa `FileVisitor`.

Rekurzivni obilazak direktorijuma

- Files.walkFileTree() pokreće rekurzivni proces obilaska strukture stabla.
 - Proces obilaska počinje od zadatog direktorijuma.

// The following code prints each file in C:\temp directory (DFS traversal).

```
Path start = Paths.get("C:\\temp");
```

```
Files.walkFileTree(start, new SimpleFileVisitor<Path>() {
```

```
    @Override
```

```
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
```

```
        System.out.println(file);
```

```
        return FileVisitResult.CONTINUE;
```

```
    }
```

```
});
```

Rekurzivni obilazak direktorijuma

- Povratna vrednost metoda visitFile koji pripada interfejsu FileVisitor ima povratnu vrednost tipa FileVisitResult.
- Moguće povratne vrednosti koje kontrolišu tok obilaska:
 - FileVisitResult.CONTINUE – nastavi obilazak.
 - FileVisitResult.TERMINATE – odmah prekida obilazak.
 - FileVisitResult.SKIP_SUBTREE – preskoči obilazak direktorijuma i njegovih poddirektorijuma (odsecanje grane u obilasku).
 - FileVisitResult.SKIP_SIBLINGS – identično kao FileVisitResult.SKIP_SUBTREE, a dodatno se preskače obilazak i preostale braće čvorova u hijerarhiji stabla.

Rekurzivni obilazak direktorijuma

```
// The following code prints Found! if file test.txt is found.  
Path start = Paths.get("C:\\temp");  
Files.walkFileTree(start, new SimpleFileVisitor<Path>() {  
    @Override  
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {  
        if (file.getFileName().toString().equals("test.txt")) {  
            System.out.println("Found!");  
            return FileVisitResult.TERMINATE;  
        }  
        return FileVisitResult.CONTINUE;  
    }  
});
```

Pretraživanje datoteka prema šablonu

- Najpre se dohvati PathMatcher objekat fajl sistema.
- Nakon toga se koristi metod matches objekta PathMatcher koji prihvata Path argument.
 - Metod vraća boolean vrednost – ili se putanja podudara sa zadatim šablonom ili to nije slučaj.

```
// matches any .java or .class file
```

```
PathMatcher matcher = FileSystems.getDefault().getPathMatcher("glob:*.{java,class}");
```

```
Path file_java = Paths.get("Test.java");
```

```
if (matcher.matches(file_java.getFileName())) // match passed
```

```
    System.out.println(file_java.getFileName());
```

```
Path file_class = Paths.get("Test.class");
```

```
if (matcher.matches(file_class.getFileName())) // match passed
```

```
    System.out.println(file_class.getFileName());
```

```
Path file_abc = Paths.get("file.abc");
```

```
if (matcher.matches(file_abc.getFileName())) // match did not pass
```

```
    System.out.println(file_abc.getFileName());
```

Ostale korisne metode

```
Path file = Paths.get("test.txt");
String type = Files.probeContentType(file);
System.out.println(type); // prints: text/plain
...
String type = Files.probeContentType(Paths.get("test.jpg"));
System.out.println(type); // prints: image/jpeg
...
FileSystem fs = FileSystems.getDefault(); // default OS file system
System.out.println(fs.getSeparator()); // prints \ or / depending on the OS
...
for (FileStore store : FileSystems.getDefault().getFileStores()) {
    System.out.println(store); // prints disks/partitions names (eg. C:, D: etc.)
}
```

Serijalizabilna klasa

```
import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L; // Serial version UID for class version control during serialization
    private String name;
    private int index;
    public Student() { }
    public Student(String name, int index) { this.name = name; this.index = index; }
    public String getName() { return name; }
    public int getIndex() { return index; }
    @Override
    public String toString() { return name + " (" + index + ")"; }
}
```

- Klasa implementira interfejs `Serializable`, što omogućava njenu serijalizaciju.
- `Serializable` je marker interfejs koji omogućava da se objekat konvertuje u bajt tok i sačuva u datoteku ili iz nje pročita, kao i da se prenese preko mreže.

Serijalizacija i deserijalizacija objekta u Javi

```
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Paths;
public class Main {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        try (ObjectOutputStream oos = new ObjectOutputStream(Files.newOutputStream(Paths.get("student.dat")))) {
            oos.writeObject(new Student("Mark", 1123));
        }
        try (ObjectInputStream ois = new ObjectInputStream(Files.newInputStream(Paths.get("student.dat")))) {
            Student student = (Student) ois.readObject(); // can throw ClassNotFoundException
            System.out.println(student);
        }
    }
}
```

Serijalizacija i deserijalizacija više objekata u Javi

```
import java.io.IOException;
import java.io.ObjectOutputStream;
import java.nio.file.Files;
import java.nio.file.Paths;

public class Main {
    public static void main(String[] args) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(Files.newOutputStream(Paths.get("students.dat"))))
        {
            oos.writeObject(new Student("Mark", 1123));
            oos.writeObject(new Student("Jane", 2233));
            oos.writeObject(new Student("Steve", 3345));
        }
    }
}
```


Serijalizacija i deserijalizacija više objekata u Javi

```
import java.io.EOFException;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.nio.file.Files;
import java.nio.file.Paths;

public class Main {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(Files.newInputStream(Paths.get("students.dat")))) {
            while (true) {
                try {
                    Student s = (Student) ois.readObject(); // can throw ClassNotFoundException
                    System.out.println(s);
                } catch (EOFException e) { break;}
            }
        }
    }
}
```

JSON – pisanje jednog objekta

```
// https://repo1.maven.org/maven2/com/google/code/gson/gson/2.10/gson-2.10.jar
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import com.google.gson.Gson;

public class Main {
    public static void main(String[] args) throws IOException {
        Gson gson = new Gson();
        Student student = new Student("Mark", 1123);
        try (BufferedWriter bw = Files.newBufferedWriter(Paths.get("student.json"))) { bw.write(gson.toJson(student)); }
    }
}

// .json file contents
// {"name":"Mark","index":1123}
```

JSON – čitanje jednog objekta

```
// https://repo1.maven.org/maven2/com/google/code/gson/gson/2.10/gson-2.10.jar
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import com.google.gson.Gson;
public class Main {
    public static void main(String[] args) throws IOException {
        Gson gson = new Gson();
        try (BufferedReader br = Files.newBufferedReader(Paths.get("student.json"))) {
            Student student = gson.fromJson(br, Student.class);
            System.out.println(student);
        }
    }
}
```

JSON – pisanje više objekata

```
// https://repo1.maven.org/maven2/com/google/code/gson/gson/2.10/gson-2.10.jar
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import com.google.gson.Gson;

public class Main {
    public static void main(String[] args) throws IOException {
        Gson gson = new Gson();
        Student[] students = { new Student("Mark", 1123), new Student("Jane", 2233) };
        try (BufferedWriter bw = Files.newBufferedWriter(Paths.get("students.json"))) { bw.write(gson.toJson(students)); }
    }
}

// .json file contents
// [
//   { "name": "Mark", "index": 1123 },
//   { "name": "Jane", "index": 2233 }
// ]
```

JSON – čitanje više objekata

```
// https://repo1.maven.org/maven2/com/google/code/gson/gson/2.10/gson-2.10.jar
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import com.google.gson.Gson;

public class Main {
    public static void main(String[] args) throws IOException {
        Gson gson = new Gson();
        try (BufferedReader br = Files.newBufferedReader(Paths.get("students.json"))) {
            Student[] students = gson.fromJson(br, Student[].class);
            for (int i = 0; i < students.length; i++)
                System.out.println(students[i]);
        }
    }
}
```

CSV – pisanje

```
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
public class Main {
    public static void main(String[] args) throws IOException {
        try (BufferedWriter bw = Files.newBufferedWriter(Paths.get("students.csv"))) {
            bw.write("name,index");
            bw.newLine();
            bw.write("Mark,1123");
            bw.newLine();
            bw.write("Ana,2233");
        }
    }
}
```

CSV – čitanje

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;

public class Main {
    public static void main(String[] args) throws IOException {
        try (BufferedReader br = Files.newBufferedReader(Paths.get("students.csv"))) {
            br.readLine(); // skip header
            String line;
            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                System.out.println(parts[0] + ": " + parts[1]);
            }
        }
    }
}
```

Rezime

- `java.io` je stariji paket koji sadrži klase za čitanje i pisanje podataka.
 - Omogućava rad sa ulaznim i izlaznim podacima kroz tokove (streams).
 - Tokovi podataka su najčešće sekvencijalni (redosledni).
 - Tokovi podataka mogu da budu bajtovski (rad sa binarnim podacima) ili karakterni (rad sa tekstualnim podacima).
 - Obezbeđuje operacije za čitanje i pisanje datoteka, filtriranje, itd.
- `java.nio.file` je moderniji API za rad sa datotekama i fajl sistemom.
 - Sadrži napredne ulazno izlazne operacije, kao i rekurzivne operacije.
 - Glavne klase ovog paketa su:
 - `Path` – predstavlja putanju do datoteke ili direktorijuma.
 - `Files` – omogućava operacije nad datotekama (kopiranje, premeštanje, brisanje, čitanje, pisanje itd.).
 - `FileSystem` – pruža informacije o fajl sistemu.

Literatura

- **dev.java/learn** - Oracle official Java learning platform; početnički nivo.
- **Head First Java** - **Kathy Sierra** (Java edukator), **Bert Bates** (Java instruktor); vizuelno učenje i analogije; početnički nivo.
- **Core Java** - **Cay Horstmann** (programer, profesor i Java autoritet); detaljno jezgro jezika Java; srednji nivo.
- **Java: The Complete Reference** - **Herbert Schildt** (programer i tehnički autor); referenca i kompletan pregled jezika Java; srednji nivo.
- **Effective Java** - **Joshua Bloch** (Java inženjer); najbolje prakse jezika Java; napredni nivo.